CNTKNW

AI Implementation Handoff

Architecture, migration, data contracts, tests, and agent operating protocol

Version 0.2 - Canonical

Updated 24 July 2026

Prepared for human review and AI-agent implementation continuity.

Version: 0.2

Audience: Claude Code, Codex, Cursor agents, Antigravity, and human developers

Owner: Lazarus Petrus

Updated: 24 July 2026

Required companion: `CNTKNW_Product_Design_Document_v0.2.md`

> **Operating rule:** Read the complete Product Design Document before modifying code. This handoff defines how to implement it. When repository reality conflicts with this handoff, inspect and report the conflict before changing architecture.

> **No false guarantee:** No handoff can ensure zero software issues. The required process is designed to minimise ambiguity, prevent architectural drift, and catch defects through small migrations, tests, build gates, and explicit acceptance criteria.

1. Mission

Build CNTKNW as a local-first personal knowledge museum and study studio with only three primary destinations: Library, Gallery, and Notes. All books and artworks open into one shared Study Workspace. The source and journal are coordinated. Content created while a source location is active inherits that context automatically.

The first engineering goal is not 3D, AI, or chess. It is a reliable local domain model that preserves source location, journal blocks, annotations, connections, workspace state, and backups.

2. Source Precedence

When instructions conflict, use this order:

1. Latest explicit user decision in the current conversation.
2. `CNTKNW_Product_Design_Document_v0.2.md`.
3. This implementation handoff.
4. Architecture Decision Records approved after v0.2.
5. Existing code behaviour.
6. Agent assumptions.

Existing code is evidence of the prototype, not the product specification.

3. Non-Negotiable Product Invariants

ID	Invariant
INV-001	Exactly three primary destinations: Library, Gallery, Notes.
INV-002	PDF reader, artwork viewer, journal, drawing, dictionary, timer, and connections are tools inside the shared Study Workspace.
INV-003	Each source has one continuous journal.
INV-004	New blocks inherit the active source location automatically.
INV-005	Empty PDF pages do not create visible empty journal sections.
INV-006	Journal, outline, cards, and map views render the same canonical blocks.
INV-007	Every explicit connection produces a backlink.
INV-008	Drawings and annotations retain editable object/vector data.
INV-009	Core reading, writing, annotation, dictionary, and search work offline.
INV-010	Manual timing is the default; quiet reading is valid focus.
INV-011	3D worlds are optional access layers and never gate content.
INV-012	Chess is a future room in Library, not a core tab.
INV-013	Schema changes always include migrations.
INV-014	UI components do not write directly to arbitrary database stores.
INV-015	Core data can be backed up and restored without cloud services.

Stop and report before implementing any change that would violate an invariant.

4. Verified Repository Snapshot

The uploaded prototype is located in `philosophy-journal-complete` and currently contains:

```
src/
  App.jsx
  db.js
  index.css
  main.jsx
  components/
    ArtGallery.jsx
    BookDetail.jsx
    BookLibrary.jsx
    ConfirmModal.jsx
    ExportImport.jsx
    NoteSystem.jsx
    PDFReader.jsx
    SessionTracker.jsx
    Settings.jsx
    Sidebar.jsx
    Toast.jsx
    Whiteboard.jsx
```

`package.json` declares:

```
{
  "react": "^19.2.7",
  "react-dom": "^19.2.7",
  "idb": "^8.0.0",
  "fabric": "^6.6.1",
  "pdfjs-dist": "^4.10.38",
  "uuid": "^11.1.0",
  "vite": "^8.1.1"
}
```

The extracted archive includes `node_modules`, but its native Vite/Rolldown optional binding is for a different or incomplete environment. The archive does not currently produce a valid build in this environment without reinstalling dependencies. Treat bundled `node_modules` as disposable.

5. Verified Prototype Problems

5.1 Architecture fragmentation

`App.jsx` uses a single `view` string to render separate mini-apps:

- library;
- book detail;
- notes;
- PDF reader;
- art gallery;
- whiteboard;
- sessions;
- export;
- settings.

This conflicts with the required three-destination architecture and shared Study Workspace.

5.2 Prop contract mismatch

`App.jsx` passes `bookId` to `PDFReader` and `NoteSystem`, but those components expect `selectedBookId`. The reader therefore does not receive its selected book through the declared prop.

5.3 Annotation store mismatch

`db.js` creates `pdfAnnotations` indexed by `pdfId` and `page`. `PDFReader.jsx` writes and reads an `annotations` store indexed by `bookId`. That store does not exist.

5.4 Note schema mismatch

`NoteSystem.jsx` stores and reads `note.text`. `PDFReader.jsx` and `ExportImport.jsx` display or export `note.content`. `BookDetail.jsx` expects `n.text`. There is no canonical note shape.

5.5 Styling inconsistency

Some components use large inline-style objects. `PDFReader.jsx` and `Whiteboard.jsx` use Tailwind-style class names, but Tailwind is not declared or configured. The resulting UI cannot have a reliable design system.

5.6 Whiteboard data loss risk

Fabric is declared but unused. The whiteboard uses a raw HTML canvas, stores history and final drawings as raster data URLs, and cannot preserve editable objects. Resizing also redraws bitmap content rather than maintaining a stable logical coordinate system.

5.7 PDF offline and resume problems

- PDF.js worker is loaded from a CDN.
- The reader initialises page state without a reliable source-specific resume contract.
- annotations are not attached to the actual PDF record consistently.
- no complete metadata extraction pipeline exists.

5.8 Backup defects

- `exportAllData()` excludes PDFs and image assets.
- `ExportImport.jsx` passes a JavaScript object directly into `Blob` instead of serialising JSON.
- import does not establish a strong parsed and schema-validated boundary.
- no manifest, migration, checksum, or conflict report exists.

5.9 Direct persistence from components

Components import generic `put`, `getAll`, and `deleteItem` and construct records independently. This guarantees schema drift as features grow.

6. Migration Strategy

Do not rewrite everything in one destructive branch and do not keep patching the current stores indefinitely.

6.1 Controlled strangler migration

1. Freeze legacy stores as read-only compatibility inputs.
2. Introduce a typed v2 domain model and repository layer.
3. Add deterministic migration functions from each legacy record shape.
4. Build the new shell and Study Workspace against v2 repositories.
5. Migrate one feature vertical at a time.
6. Validate counts, IDs, binaries, and relationships.
7. Keep a rollback backup until migration is verified.
8. Remove legacy components only after replacement acceptance tests pass.

6.2 Never mutate v1 data in place

Migration should write to new stores or a new database version. Preserve the original database until the user confirms a successful migration and backup.

6.3 Suggested legacy mapping

Legacy	v2 target	Notes
`books`	`sources` + `books`	Generate stable source ID; retain legacy ID in migration metadata.

Legacy	v2 target	Notes
`pdfs`	`assets` + `sourceAssets`	Hash binary where practical; retain original filename and MIME type.
`notes`	`journals` + `journalBlocks`	Accept both `text` and `content`; infer block type and source page.
`pdfAnnotations`	`annotations`	Map `pdfId`; normalise geometry if available.
broken `annotations` references	recovery report	Scan only if store exists in a real user DB.
`drawings`	`drawingDocuments` + block refs	Raster-only data becomes a locked legacy image block, not fake editable vectors.
`sessions`	`studySessions`	Map book to source and preserve duration.
`artworks`	`sources` + `artworks` + assets	Move analysis fields into journal blocks where appropriate.
`appState`	`settings` + `workspaceStates`	Split global settings from source-specific resume state.

7. Recommended Technology Direction

The agent must verify current package compatibility before installation. Do not assume version numbers from memory.

7.1 Baseline

- React with TypeScript;
- Vite;
- React Router for real URLs and nested layouts;
- Zod for runtime schema validation;
- Dexie or a similarly migration-friendly IndexedDB wrapper for web storage;
- repository and service interfaces that can later target SQLite/Tauri;
- Tiptap/ProseMirror technical spike for the block journal;
- PDF.js with locally bundled worker;
- `@xyflow/react` or equivalent for mind/argument maps;
- an editable vector library after a focused comparison spike;
- Zustand for transient UI state only, not as the canonical database;
- MiniSearch/FlexSearch or another local index behind a search interface;
- Vitest and React Testing Library;
- Playwright for critical end-to-end flows;
- ESLint, TypeScript strict mode, and a formatter.

7.2 Why repository abstraction matters

Browser IndexedDB is acceptable for an early web release. A large personal library may later benefit from Tauri, a user-selected folder, SQLite, and filesystem assets. Domain services must not depend directly on Dexie so the persistence adapter can change.

7.3 Libraries that must remain later-phase

- React Three Fiber/Drei for immersive worlds;
- `chess.js` and Stockfish WASM for the Chess Room;
- OCR engine;
- AI SDKs;
- sync or collaboration frameworks.

Do not install them during Phase 0 unless a small isolated spike is explicitly requested.

8. Target Repository Structure

```
src/
  app/
    App.tsx
    router.tsx
    providers.tsx
    routes/
```

```
      LibraryRoute.tsx
      GalleryRoute.tsx
      NotesRoute.tsx
      StudyWorkspaceRoute.tsx
  domain/
    sources/
      source.types.ts
      source.schemas.ts
      source.service.ts
    journals/
      journal.types.ts
      journal.schemas.ts
      journal.service.ts
    annotations/
    connections/
    concepts/
    glossary/
    sessions/
    maps/
    search/
  data/
    db/
      database.ts
      migrations/
      schemaVersions.ts
    repositories/
      SourceRepository.ts
      JournalRepository.ts
      AnnotationRepository.ts
      ConnectionRepository.ts
      SessionRepository.ts
      AssetRepository.ts
    adapters/
      indexeddb/
      memory/
  features/
    library/
    gallery/
    study-workspace/
      source-pane/
      journal-pane/
      workspace-toolbar/
    pdf/
    artwork-viewer/
    journal-editor/
    annotations/
    dictionary/
    connections/
    maps/
    sessions/
    notes/
  components/
    ui/
    layout/
  design/
    tokens.css
    typography.css
    motion.css
  hooks/
  lib/
  tests/
    fixtures/
    factories/
  legacy/
    v1-readers/
    v1-migration/
```

Feature folders may import domain services and UI components. They must not import adapter implementation details directly.

9. Canonical TypeScript Contracts

These are conceptual contracts. Keep names stable unless an approved ADR changes them.

9.1 Shared primitives

```
type EntityId = string;
type ISODateTime = string;
type Revision = number;

type NormalizedPoint = { x: number; y: number };
type NormalizedRect = { x: number; y: number; width: number; height: number };

type TextAnchor = {
  page: number;
  startOffset?: number;
  endOffset?: number;
  quote: string;
  prefix?: string;
  suffix?: string;
};
```

Use UUIDv7 or ULID after one decision. Do not combine timestamp strings, book IDs, and page numbers as record IDs.

9.2 Source

```
type SourceType = 'book' | 'artwork' | 'chess-game' | 'chess-position';

type Source = {
  id: EntityId;
  type: SourceType;
  title: string;
  subtitle?: string;
  creatorIds: EntityId[];
  collectionIds: EntityId[];
  tagIds: EntityId[];
  archivedAt?: ISODateTime;
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
  legacy?: { database: string; recordId: string };
};

type BookDetails = {
  sourceId: EntityId;
  authorText?: string;
  language?: string;
  publicationDate?: string;
  pageCount?: number;
  isbn?: string;
  subject?: string;
  currentStatus: 'unread' | 'reading' | 'paused' | 'completed';
};

type ArtworkDetails = {
  sourceId: EntityId;
  artistText?: string;
  creationDate?: string;
  period?: string;
  movement?: string;
  medium?: string;
  dimensions?: string;
  culture?: string;
  placeCreated?: string;
  currentLocation?: string;
};
```

9.3 Asset

```
type Asset = {
  id: EntityId;
  kind: 'pdf' | 'image' | 'thumbnail' | 'dictionary-pack' | 'legacy-raster';
  mimeType: string;
  originalName?: string;
  sizeBytes: number;
```

```
  checksum?: string;
  storageKey: string;
  createdAt: ISODateTime;
};

type SourceAsset = {
  sourceId: EntityId;
  assetId: EntityId;
  role: 'primary' | 'cover' | 'thumbnail' | 'supplement';
};
```

Binary data does not belong inside metadata records as base64 unless a small temporary import boundary requires it.

9.4 Source location

```
type SourceLocation = {
  sourceType: SourceType;
  sourceId: EntityId;
  pdf?: {
    page: number;
    textAnchor?: TextAnchor;
  };
  artwork?: {
    region?: NormalizedRect;
  };
  chess?: {
    moveNumber?: number;
    fen?: string;
  };
};
```

PDF pages are one-based at the domain/UI boundary. If PDF.js uses zero-based indexes internally, convert in one adapter and test it.

9.5 Workspace state

```
type WorkspaceState = {
  id: EntityId;
  sourceId: EntityId;
  activeLocation: SourceLocation;
  sourceView: {
    mode: 'continuous' | 'single' | 'spread' | 'artwork';
    zoom: number;
    scrollOffset?: number;
  };
  journalView: {
    scrollOffset: number;
    activeBlockId?: EntityId;
  };
  layout: {
    mode: 'split' | 'source-focus' | 'journal-focus' | 'compare';
    splitRatio: number;
    sourceCollapsed: boolean;
    journalCollapsed: boolean;
  };
  openPanel?: 'dictionary' | 'connections' | 'search' | 'contents';
  updatedAt: ISODateTime;
  revision: Revision;
};
```

9.6 Journal and blocks

```
type Journal = {
  id: EntityId;
  sourceId?: EntityId;
  title: string;
  kind: 'source' | 'standalone';
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};
```

```
type BlockKind =
  | 'paragraph'
  | 'heading'
  | 'quote'
  | 'question'
  | 'claim'
  | 'evidence'
  | 'objection'
  | 'definition'
  | 'drawing'
  | 'argument-map'
  | 'image'
  | 'connection-embed'
  | 'divider'
  | 'callout';

type JournalBlock = {
  id: EntityId;
  journalId: EntityId;
  kind: BlockKind;
  sourceLocation?: SourceLocation;
  parentBlockId?: EntityId;
  rank: string;
  content: BlockContent;
  conceptIds: EntityId[];
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
  provenance: 'user' | 'imported' | 'migration' | 'ai-draft';
};
```

`BlockContent` must be a discriminated union validated by Zod. Never use a single optional-field object for every block type.

Example:

```
type ParagraphContent = { type: 'paragraph'; doc: ProseMirrorJSON };
type QuoteContent = {
  type: 'quote';
  quote: string;
  anchor?: TextAnchor;
  commentary?: ProseMirrorJSON;
};
type DefinitionContent = {
  type: 'definition';
  term: string;
  dictionaryEntryId?: EntityId;
  sourceMeaning?: string;
  personalMeaning?: string;
};
```

9.7 Annotation

```
type Annotation = {
  id: EntityId;
  sourceId: EntityId;
  assetId: EntityId;
  page: number;
  kind: 'highlight' | 'underline' | 'strike' | 'pen' | 'rect' | 'ellipse' | 'arrow' | 'text' | 'region' |
'bookmark';
  geometry: unknown; // discriminated and normalised by kind
  style: AnnotationStyle;
  textAnchor?: TextAnchor;
  linkedBlockIds: EntityId[];
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};
```

Coordinates are normalised to the unrotated PDF page space. Store page rotation separately and transform only in the rendering adapter.

9.8 Connection

```
type Reference =
  | { type: 'source'; id: EntityId }
  | { type: 'location'; location: SourceLocation }
  | { type: 'block'; id: EntityId }
  | { type: 'annotation'; id: EntityId }
  | { type: 'concept'; id: EntityId }
  | { type: 'glossary-entry'; id: EntityId };

type Connection = {
  id: EntityId;
  from: Reference;
  to: Reference;
  relationship: string;
  direction: 'directed' | 'undirected';
  note?: string;
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};
```

Backlinks are queried from Connection records. Do not store a second mutable backlinks array on every entity.

9.9 Concepts and glossary

```
type Concept = {
  id: EntityId;
  title: string;
  summary?: string;
  aliases: string[];
  parentConceptIds: EntityId[];
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};

type GlossaryEntry = {
  id: EntityId;
  term: string;
  language: string;
  dictionaryPackId?: EntityId;
  dictionarySenseIds: string[];
  sourceMeanings: Array<{ sourceId: EntityId; meaning: string }>;
  personalMeaning?: string;
  occurrenceLocations: SourceLocation[];
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};
```

9.10 Mind maps

```
type MapNodeRef =
  | { type: 'block'; id: EntityId }
  | { type: 'concept'; id: EntityId }
  | { type: 'source'; id: EntityId }
  | { type: 'location'; location: SourceLocation };

type MindMapView = {
  id: EntityId;
  title: string;
  kind: 'mind-map' | 'argument-map' | 'free-canvas';
  nodes: Array<{
    id: EntityId;
    ref: MapNodeRef;
    x: number;
    y: number;
    width?: number;
    height?: number;
    collapsed?: boolean;
  }>;
```

```
  edges: Array<{
    id: EntityId;
    connectionId?: EntityId;
    sourceNodeId: EntityId;
    targetNodeId: EntityId;
    viewRelationship?: string;
  }>;
  viewport: { x: number; y: number; zoom: number };
  createdAt: ISODateTime;
  updatedAt: ISODateTime;
  revision: Revision;
};
```

Map nodes reference canonical entities. Do not copy full block text into map storage.

9.11 Sessions

```
type StudySession = {
  id: EntityId;
  sourceId: EntityId;
  mode: 'manual' | 'pomodoro-25' | 'deep-50' | 'custom' | 'stopwatch';
  startedAt: ISODateTime;
  endedAt?: ISODateTime;
  focusedSeconds: number;
  breakSeconds: number;
  manualPauseSeconds: number;
  removedIdleSeconds: number;
  startLocation: SourceLocation;
  endLocation?: SourceLocation;
  visitedLocations: SourceLocation[];
  createdBlockIds: EntityId[];
  createdAnnotationIds: EntityId[];
  createdConnectionIds: EntityId[];
  dictionaryLookupIds: EntityId[];
  reflection?: string;
  revision: Revision;
};
```

10. Repository Interfaces

Define interfaces before adapters.

```
interface JournalRepository {
  getJournalForSource(sourceId: EntityId): Promise<Journal | null>;
  listBlocks(journalId: EntityId): Promise<JournalBlock[]>;
  createBlock(input: CreateBlockInput): Promise<JournalBlock>;
  updateBlock(input: UpdateBlockInput): Promise<JournalBlock>;
  moveBlock(input: MoveBlockInput): Promise<void>;
  softDeleteBlock(blockId: EntityId): Promise<void>;
  transact<T>(operation: () => Promise<T>): Promise<T>;
}
```

Add equivalent repositories for sources, assets, workspace state, annotations, connections, concepts, glossary, maps, sessions, settings, and backup manifests.

Rules:

- repository methods accept validated domain inputs;
- adapters map storage records to domain records;
- components call use cases/services, not repositories directly where business rules exist;
- generic `put(storeName, object)` is prohibited outside migration tooling;
- transactions cross related records through a unit-of-work boundary.

11. Core Use Cases

Implement and test use cases independently from UI.

11.1 Import PDF

```
select file
-> validate MIME/signature and size
-> create asset
-> parse metadata/page count/outline
-> create or attach Book source
-> create source journal
-> create initial workspace state
-> queue text indexing and cover generation
-> open workspace
```

Failure after binary storage must roll back metadata or mark an import recovery record.

11.2 Create page-linked block

```
active workspace location = book A, page 5
user starts typing
-> ensure source journal exists
-> create paragraph block with page 5 SourceLocation
-> insert rank after current block or at end of page group
-> persist
-> update journal view
```

The UI does not ask for the page. The use case receives it from WorkspaceContext.

11.3 Capture quote

```
selected PDF text
-> build resilient TextAnchor with quote/prefix/suffix
-> create quote block at active location
-> optionally create highlight annotation in same transaction
-> maintain reciprocal block/annotation references
```

11.4 Create connection

```
from reference + to reference + relationship
-> validate both endpoints
-> create one Connection record
-> invalidate backlink queries for both endpoints
-> update map/search indexes asynchronously
```

11.5 Resume source

```
open source
-> load WorkspaceState
-> load source asset and source details
-> render saved location
-> load journal blocks near active context
-> restore panel state
-> mark recently opened
```

Never set page to 1 after loading a valid saved state.

12. Workspace Context

Create one typed WorkspaceContext or equivalent controller that owns:

- source ID and details;
- asset ID;
- active SourceLocation;
- current text selection;
- workspace layout;
- active tool;
- current session;

- save state;
- commands for creating contextual content.

Journal, dictionary, annotation, and connection features consume this context. They must not each maintain independent page state.

13. Editor Architecture

13.1 Technical spike before commitment

Time-box a spike comparing at least two viable editors against these requirements:

- custom block/node types;
- stable block IDs;
- JSON document storage;
- collaboration not required now;
- source-location metadata per top-level block;
- drag/reorder;
- nested blocks;
- keyboard navigation;
- paste handling;
- custom React node views;
- large-document performance;
- history and recovery;
- accessibility.

Tiptap/ProseMirror is the recommended baseline to test first.

13.2 Page grouping

Do not create one editor document per PDF page. That would fragment the journal and make cross-page writing difficult.

Recommended approach:

- one source journal;
- top-level blocks have source locations;
- UI groups visible blocks by page/location;
- virtualise page groups in review mode;
- active page group opens or is created lazily;
- blocks may be manually moved to another page with an explicit action.

13.3 Autosave

- debounce ordinary text writes;
- flush on blur, page navigation, source close, visibility change, and before unload where possible;
- retain local editor history between saves;
- use optimistic concurrency revision checks;
- never silently overwrite a newer revision;
- surface recoverable conflicts.

14. PDF Architecture

14.1 Worker

Bundle PDF.js worker as a local build asset. CDN worker URLs violate offline requirements.

14.2 Rendering

- virtualise continuous pages;
- render visible and adjacent pages only;
- cache page dimensions and thumbnails;
- keep text, annotation, and canvas layers separate;

- apply zoom through deterministic coordinate transforms;
- cancel obsolete render tasks;
- move heavy extraction/indexing to a worker.

14.3 Text anchors

Text positions may shift across PDF.js versions or extraction differences. Store:

- page;
- quote text;
- prefix and suffix context;
- offsets where available.

On reopen, resolve by offsets first and quote-context matching second. Mark unresolved anchors rather than guessing silently.

14.4 Annotation coordinates

Use page-space normalised coordinates. Add unit tests for:

- zoom in/out;
- rotated pages;
- viewport resize;
- continuous/single mode;
- reopening;
- high-DPI screens.

15. Drawing and Canvas Architecture

15.1 Separate purposes

- PDF annotation overlay: constrained to a PDF page coordinate system.
- Drawing block: embedded editable canvas inside the journal.
- Mind/argument map: graph of canonical refs and typed edges.
- Free canvas: spatial composition of canonical refs, frames, images, and drawing elements.

Do not force one library to solve every mode if it weakens data fidelity.

15.2 Persistence

Store a versioned object document. Cache PNG/SVG previews for lists and export. Never make the preview the only source.

15.3 Undo

Use operation history or library-native object history. Do not snapshot the full canvas as a base64 bitmap on every pointer-up event.

16. Search Architecture

Define a `SearchService` with replaceable index adapters.

Index documents derived from canonical entities:

- source metadata;
- block plain text;
- concepts and aliases;
- glossary terms and meanings;
- PDF extracted pages;
- annotation text;
- connection relationships.

Index updates are asynchronous but observable. The app can rebuild the entire index from canonical storage. Index data is excluded from mandatory backups or marked disposable.

17. Backup Format

Use a versioned manifest.

```
{
  "format": "cntknw-portable-library",
  "version": 1,
  "createdAt": "ISO-8601",
  "appVersion": "...",
  "schemaVersion": 2,
  "records": {
    "sources": "data/sources.jsonl",
    "journalBlocks": "data/journal-blocks.jsonl"
  },
  "assets": [
    { "id": "...", "path": "assets/...pdf", "checksum": "..." }
  ]
}
```

Requirements:

- serialise explicitly;
- validate every imported file;
- verify checksums where supplied;
- produce a dry-run import report;
- state added, updated, skipped, and conflicting records;
- never overwrite newer local content without explicit resolution;
- test restoration into an empty database.

18. Design System Implementation

18.1 Tokens first

Create CSS custom properties for:

- colour roles;
- typography;
- spacing;
- radius;
- border;
- shadow;
- motion duration/easing;
- z-index layers;
- panel and reading widths.

Do not mix unconfigured utility classes with large inline-style objects.

18.2 Component primitives

Implement and document:

- Button;
- IconButton;
- Input, Select, Textarea;
- Dialog and ConfirmDialog;
- Popover and Tooltip;
- CommandPalette;
- Tabs for local subviews only;
- SplitPane;
- Surface/Card;
- Toast/Status;
- EmptyState;
- Menu;
- ScrollArea;
- ResizablePanel;

- FocusRing.

18.3 Story and state coverage

Each primitive should demonstrate default, hover, focus, active, disabled, loading, error, and reduced-motion behaviour. Storybook is optional; a local component-lab route is sufficient initially.

19. Exact Build Order

Phase 0A - Clean reproducible build

1. Delete bundled `node_modules` from the working copy.
2. Use the package manager selected by the owner and generate a clean lock file.
3. Add TypeScript and strict configuration.
4. Add lint, unit-test, and E2E scripts.
5. Confirm clean install and build in a new directory or CI environment.

Gate: install, typecheck, lint, unit tests, and production build succeed.

Phase 0B - Shell and routing

1. Add real router.
2. Build Library, Gallery, Notes layouts.
3. Add nested Study Workspace route such as `/study/:sourceId`.
4. Create design tokens and primitives.
5. Remove separate top-level reader/whiteboard/session routes from navigation.

Gate: three primary destinations only; route refresh works.

Phase 0C - Domain and persistence

1. Add Zod schemas and TypeScript contracts.
2. Implement repositories and in-memory test adapters.
3. Implement v2 IndexedDB adapter with migrations.
4. Add legacy v1 readers and migration dry run.
5. Add transaction and error reporting.

Gate: repository contract tests pass against memory and IndexedDB adapters.

Phase 1 - PDF vertical slice

1. import one PDF;
2. create source and asset;
3. render with local worker;
4. navigate pages;
5. persist and restore workspace state;
6. extract metadata/page count;
7. add lazy page rendering and basic text layer.

Gate: AC-001 and AC-002 pass in Playwright offline mode.

Phase 2 - Journal vertical slice

1. source journal creation;
2. contextual paragraph block;
3. page grouping;
4. autosave and recovery;
5. quote block from PDF selection;
6. complete journal review.

Gate: AC-003 and AC-004 pass; no `text`/`content` ambiguity exists.

Phase 3 - Annotation and drawing

1. highlight and text anchor;
2. normalised geometry;
3. pen/shapes overlay;
4. editable journal drawing block;
5. save/reopen/zoom tests.

Gate: AC-007 and AC-008 pass.

Phase 4 - Connections and concepts

1. generic Reference union;
2. Connection service;
3. backlinks;
4. concept cards;
5. page-to-page and cross-source linking.

Gate: AC-005 passes.

Phase 5 - Dictionary

1. choose licensed dataset;
2. pack importer and local index;
3. selection lookup;
4. save glossary entry;
5. personal/source meanings;
6. occurrence navigation.

Gate: AC-006 passes with network disabled.

Phase 6 - Notes, search, and maps

1. unified Notes queries;
2. local search index;
3. journal/outline/cards projections;
4. mind-map view referencing canonical blocks;
5. argument edge semantics;
6. accessible outline.

Gate: AC-009, AC-010, and AC-015 pass.

Phase 7 onward

Continue with artwork, sessions, production hardening, immersive worlds, then Chess Room exactly in the Product Design Document order.

20. Testing Strategy

20.1 Unit tests

Test pure functions and schemas:

- source-location equality and grouping;
- PDF page index conversion;
- normalised coordinate transforms;
- rank ordering and block movement;
- connection endpoint validation;
- backlink queries;
- session duration accounting;
- migration mapping;

- import manifest validation;
- text-anchor resolution;
- stats aggregation.

20.2 Repository contract tests

Run the same suite against:

- in-memory adapter;
- IndexedDB adapter using a browser test environment.

Test transactions, revision conflicts, soft deletion, migration, and failed writes.

20.3 Component tests

- source context propagation;
- block menu and transformations;
- save status;
- dictionary popup;
- connection creator;
- page group rendering;
- keyboard navigation;
- reduced motion;
- focus restoration after dialogs.

20.4 End-to-end tests

Critical Playwright journeys:

1. import PDF offline, write on page 5, close, reopen, confirm page and block;
2. navigate to page 100, create connection, verify backlinks at both ends;
3. select word, define offline, save meaning, jump to original passage;
4. highlight text, zoom, resize, switch view, reopen, verify alignment;
5. create drawing, edit objects after reopen;
6. change journal block from map, verify journal update and no duplicate;
7. start manual session, wait without interaction, verify timer continues;
8. export full backup, clear test DB, restore, verify binaries and records;
9. use keyboard only for import, reading, writing, and search;
10. run core flow with network requests blocked.

20.5 Migration tests

Maintain representative v1 fixtures containing:

- note using `text`;
- note using `content`;
- PDF and book records;
- raster drawing;
- sessions;
- artwork analysis;
- missing or malformed fields.

The migration report must be deterministic and count every source record as migrated, skipped with reason, or failed.

21. Performance Budgets

Initial targets to validate on a typical mid-range laptop:

- launch shell usable without loading all assets;
- typing remains responsive during autosave and indexing;
- continuous PDF mode renders visible and adjacent pages only;
- opening a book does not load the complete journal into one DOM tree when extremely large;
- maps virtualise or simplify distant nodes;

- binary files are not repeatedly converted to base64;
- background workers handle PDF extraction, indexing, and heavy analysis;
- memory use is profiled with a large synthetic library.

Do not hard-code performance claims without measurement. Add benchmarks and record the test device.

22. Error and Recovery Rules

Every durable command returns a typed result or throws a typed domain error.

User-visible categories:

- storage unavailable or full;
- asset missing;
- unsupported or corrupt PDF;
- write conflict;
- dictionary pack unavailable;
- backup validation failed;
- migration incomplete;
- search index rebuilding.

Never use `console.error` as the only failure handling. Preserve unsaved editor state and provide a recovery action.

23. Security and Privacy

- do not upload user sources or notes without explicit opt-in;
- do not add analytics by default;
- do not log source text or journal content to remote services;
- validate imported filenames and archive paths;
- prevent zip-slip during backup import;
- sanitise rich-text HTML and pasted content;
- use Content Security Policy compatible with bundled workers;
- document any third-party dictionary, OCR, AI, or sync data flow;
- keep secrets out of frontend bundles.

24. AI Feature Guardrails

AI is not part of the initial core. When added:

- generated text is labelled as an AI draft;
- source quotes remain distinct from generation;
- the user explicitly accepts content into a journal;
- local and cloud modes disclose data handling;
- the model receives only the context the user selects;
- AI cannot silently create hundreds of links or concepts;
- every generated connection exposes its basis;
- AI content can be removed without damaging user-authored records.

25. Agent Operating Protocol

Before coding:

1. Read both v0.2 Markdown documents fully.
2. Inspect the repository and report actual files, dependencies, and build status.
3. State the phase, requirement IDs, and acceptance criteria being implemented.
4. Identify assumptions and conflicts.
5. Create a small implementation plan.

During coding:

1. Keep changes within the active phase.
2. Add or update tests with the implementation.
3. Do not invent new domain shapes inside components.
4. Use migrations for schema changes.
5. Run typecheck, lint, unit tests, relevant E2E tests, and build.
6. Preserve user data and legacy fixtures.
7. Update documentation when an approved decision changes.

After coding:

1. Summarise files changed.
2. Report test commands and exact outcomes.
3. Map completed work to requirement IDs.
4. List known limitations and deferred items.
5. Do not claim completion when a gate is failing.

26. Prohibited Shortcuts

- No new main tabs.
- No direct component calls such as `put('notes', arbitraryObject)`.
- No single untyped `Note` record shared by unrelated features.
- No storing annotations by `bookId` when the actual PDF asset is required.
- No page-1 reset after async load.
- No CDN worker for PDF.js.
- No flattened-only drawing persistence.
- No canvas nodes containing independent copies of journal text.
- No 3D catalogue with a second book model.
- No `any` to bypass core domain typing.
- No deleting v1 data before verified migration and backup.
- No unvalidated JSON import.
- No asynchronous work hidden behind optimistic success messages.
- No fake AI explanations presented as dictionary or source content.

27. Definition of Done

A task is done only when:

- product invariants remain intact;
- relevant acceptance criteria pass;
- TypeScript strict checks pass;
- lint passes;
- unit and repository tests pass;
- relevant Playwright journeys pass;
- production build passes from a clean install;
- offline behaviour is tested when relevant;
- keyboard and focus behaviour is checked;
- persistence survives reload;
- schema changes have migrations;
- errors have user-visible recovery;
- documentation and requirement status are updated;
- no hidden console error remains in the tested flow.

28. Checkpoint Report Template

```
Phase:
Requirements implemented:
Acceptance criteria:
```

```
Repository observations:
- ...

Changes:
- ...

Data/schema changes:
- migration version ...
- rollback/recovery ...

Verification:
- install:
- typecheck:
- lint:
- unit tests:
- E2E tests:
- production build:
- offline test:

Known limitations:
- ...

Next safe step:
- ...
```

29. Copy-Paste Master Prompt for a Coding Agent

```
You are implementing CNTKNW, a local-first personal knowledge museum and study studio.

Before editing code, read these files completely:
1. CNTKNW_Product_Design_Document_v0.2.md
2. CNTKNW_AI_Implementation_Handoff_v0.2.md
3. CNTKNW_Requirements_v0.2.json

Then inspect the actual repository. Do not assume the existing code matches the product specification.

Non-negotiable rules:
- The only primary destinations are Library, Gallery, and Notes.
- Books and artworks open in one shared Study Workspace.
- Each source has one continuous journal.
- New journal blocks inherit the active source location automatically.
- Empty pages do not create visible journal sections.
- Journal, outline, cards, and maps render the same canonical blocks.
- Explicit links are first-class Connection records with backlinks.
- Editable drawings and annotations may not be stored only as flattened images.
- Core reading, writing, annotation, dictionary, and search are offline-capable.
- Components do not write arbitrary records directly to database stores.
- Every schema change includes a migration.
- Do not work on immersive 3D or chess before the core phases pass.

For this task:
1. State the current phase and requirement IDs.
2. Audit the relevant existing files.
3. Identify conflicts or missing information.
4. Implement the smallest complete vertical slice.
5. Add tests before claiming completion.
6. Run clean install/build, typecheck, lint, unit tests, relevant E2E tests, and offline checks.
7. Report exact results and known limitations.

Never hide failures, invent completed work, or weaken a requirement silently.
```

30. First Assignment for the Agent

The first implementation assignment should be **Phase 0A-0C only**:

1. create a clean branch and reproducible dependency install;

2. migrate the app shell to TypeScript;
3. establish Library, Gallery, Notes, and nested Study Workspace routing;
4. create design tokens and basic UI primitives;
5. define Zod schemas and repository interfaces;
6. create memory and IndexedDB adapters;
7. implement a read-only v1 audit and migration dry-run report;
8. add unit, repository, and basic route tests;
9. do not yet build the full journal, 3D, chess, or AI.

The output of the first assignment must include an architecture report and a tested migration plan before any destructive data operation.

31. Handoff Change Log

Version	Date	Change
0.2	24 July 2026	Initial implementation handoff based on the v0.2 product design and verified uploaded prototype audit.